

Generative Augmentation in Butterfly Identification

Matilde Carvalho¹[2022231937] and Rita Duarte¹[1111–2222–3333–4444]

Master’s degree in Biomedical Engineering, Physics Department, University of
Coimbra

Abstract. The abstract should briefly summarize the contents of the paper in 150–250 words.

Keywords: Fine-Grained Classification · Generative Augmentation · Image Classification.

1 Introduction

The recent breakthroughs in Deep Learning, particularly through Convolutional Neural Networks (CNNs), have drastically transformed the field of Computer Vision, enabling automated systems to achieve human-like accuracy in complex image classification tasks. However, the high performance of these deep models is intrinsically tied to the availability of massive, diverse, and well-annotated datasets, but collecting a substantial volume of high-quality samples is often challenging, costly, and time-consuming. When trained on limited data, deep architectures are highly susceptible to overfitting, which severely undermines their capacity to generalize to unseen scenarios. To mitigate the issues arising from data scarcity, practitioners traditionally rely on standard data augmentation techniques, such as geometric transformations, color jittering, and random erasing. While effective at forcing the network to learn invariant features, these traditional methods merely manipulate existing samples without introducing truly novel semantic content or underlying structural variations. Recently, Deep Generative Models, such as Autoencoders (AEs), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs), have emerged as a powerful alternative. By learning the underlying probability distribution of the training data, these models can synthesize entirely new, realistic samples, providing a more robust framework for data augmentation. This report documents the development and evaluation of a generative data augmentation pipeline, developed within the scope of the Advanced Machine Learning course. The primary objective of this project is to improve the generalization capabilities of a fixed baseline classifier tasked with identifying 75 distinct species of butterflies. Under a strict operational constraint where the classifier’s architecture and training hyperparameters must remain unaltered, our approach focuses exclusively on expanding the training distribution. We explore and compare the effectiveness of different generative paradigms in creating artificial samples that can successfully guide the classifier toward more robust decision boundaries.

2 Problem Description

The dataset utilized in this project consists of 5199 labeled images of butterflies representing 75 distinct classes. This fine-grained classification task exhibits high intra-class variance, characterized by diverse angles, complex natural backgrounds, and varying lighting conditions, coupled with low inter-class variance, featuring morphologically identical wing patterns across different species. The primary challenge relies on synthesizing artificial data capable of expanding this distribution without introducing semantic confusion between closely related categories.

2.1 Baseline Classifier Architecture

The predetermined model consists of a feature extractor and a fully-connected classifier head. The feature extractor comprises three sequential convolutional blocks, where each individual block is mathematically defined as:

$$\begin{aligned} \text{Block}(C_{in}, C_{out}) = & \text{Conv2d}_{3 \times 3} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \\ & \text{Conv2d}_{3 \times 3} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}_{2 \times 2} \end{aligned} \quad (1)$$

The feature maps progress through channel sizes of $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$, effectively reducing the spatial resolution from 64×64 down to 8×8 . Finally, an `AdaptiveAvgPool2d((1, 1))` layer collapses the remaining spatial dimensions, outputting a 256-dimensional feature vector.

The classifier head is designed as a deep multi-layer perceptron (MLP) with three intermediate hidden layers and dropout regularization to mitigate overfitting. Given the initial feature vector $x_0 \in \mathbb{R}^{256}$, the first hidden layer maps the features to a higher dimension, computed as $x_1 = \text{Dropout}_{0.5}(\text{ReLU}(\text{Linear}(256 \rightarrow 512)(x_0)))$. This representation is sequentially projected through wider spaces, where the second layer yields $x_2 = \text{Dropout}_{0.5}(\text{ReLU}(\text{Linear}(512 \rightarrow 1024)(x_1)))$, and the third layer produces $x_3 = \text{Dropout}_{0.5}(\text{ReLU}(\text{Linear}(1024 \rightarrow 2048)(x_2)))$. Lastly, a final linear transformation scales the hidden features to match the class distribution, outputting the final raw scores as logits = $\text{Linear}(2048 \rightarrow 75)(x_3)$.

3 Approach

This section details the generative frameworks implemented to synthesize training samples. Our methodology focuses on the Variational Autoencoder (VAE) paradigm, evaluating a standard baseline and an advanced structural variant designed to bypass classical pixel-level limitations [?].

3.1 Generative Architectures

The baseline VAE processes $64 \times 64 \times 3$ inputs. The Encoder features four convolutional layers with LeakyReLU activations (slope 0.2) and Batch Normalization, halving dimensions via stride 2 down to a $256 \times 4 \times 4$ tensor. This tensor is flattened and projected to latent mean μ and log-variance $\log(\sigma^2)$ vectors in $\mathbb{R}^{128}$. Backpropagation uses the reparameterization trick: $z = \mu + \epsilon \odot \sigma$, where $\epsilon \sim \mathcal{N}(0, I)$. The Decoder mirrors this topology, expanding codes back to feature space via a dense layer and four transposed convolutional layers (`ConvTranspose2d`) with ReLU and Batch Normalization, terminating in a `Tanh` activation. The baseline objective optimizes pixel-level Mean Squared Error (MSE) reconstruction and Kullback-Leibler (KL) divergence:

$$\mathcal{L}_{\text{VAE}} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|_2^2 + \beta \cdot D_{\text{KL}}(q_\phi(z|x) \| p(z)) \quad (2)$$

An improved VAE variant addresses pixel-level blurriness via VGG-based perceptual loss and KL annealing. A frozen, pre-trained VGG-16 extracts features to compute the MSE distance across layers `relu1_2`, `relu2_2`, and `relu3_3`:

$$\mathcal{L}_{\text{VGG}}(x, \hat{x}) = \sum_{j \in \{1_2, 2_2, 3_3\}} \frac{1}{N_j} \|\Phi_j(x) - \Phi_j(\hat{x})\|_2^2 \quad (3)$$

forcing the model to prioritize high-level morphology over pixel alignment. Concurrently, a linear scheduler scales β from 0.0 to 0.5 during the first 15 epochs to prevent posterior collapse. The total joint loss is:

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{MSE}}(x, \hat{x}) + \lambda_{\text{VGG}} \mathcal{L}_{\text{VGG}}(x, \hat{x}) + \beta(t) D_{\text{KL}}(q_\phi(z|x) \| p(z)) \quad (4)$$

where the perceptual weight is fixed at $\lambda_{\text{VGG}} = 0.5$.

3.2 Conditional Perturbation and Data Integration

To enable class-conditional synthesis from unconditional VAEs, we implement latent space perturbation around real samples. For each of the 75 classes, real images are encoded to extract their parameters (μ, σ) . To synthesize new images with semantic fidelity and spatial variance, latent codes are perturbed across three scales ($z_{\text{synth}} = \mu + \alpha \cdot \sigma \cdot \eta$), where $\alpha \in \{0.3, 0.6, 1.0\}$ and $\eta \sim \mathcal{N}(0, I)$. The decoder processes these vectors to generate 75 synthetic images per class. A PyTorch `ConcatDataset` merges the 4159 training images with the 5625 synthesized samples, expanding the final training set to 9784 images (135.2% increase), while keeping the 1040 validation samples strictly isolated.

4 Experimental Setup

To ensure reproducibility and an unbiased benchmark, this section outlines the data partitioning strategy, parameter configurations, and evaluation metrics [?].

4.1 Data Splitting and Protocols

The original dataset (`train.csv`) was partitioned using stratified sampling to maintain identical class proportions across subsets [?]. A training set (80%, 4159 images) was allocated for weight optimization, while a validation set (20%, 1040 images) was isolated to monitor generalization, manage hyperparameter grid-searches, and trigger early stopping. No validation samples were exposed to the models during backpropagation. The partition was locked using a global random seed ($GLOBAL_SEED = 42$).

To ensure fairness, the optimization protocol for the classifier was kept completely static across all experiments (raw data baseline, baseline VAE, and improved VAE expansion). The architectural configuration, loss formulation, and optimization hyper-parameters of the `BaselineCNN` remained immutable.

4.2 Hyperparameters and VAE Training Phases

The development of the VAE generator progressed through five incremental phases, with optimization configurations summarized alongside the classifier in Table 1.

Phase 1 — Baseline VAE: Trained using standard configurations ($latent_dim = 128$, $\beta = 1.0$, $\eta = 10^{-3}$) to establish a baseline benchmark. Early stopping triggered convergence at epoch 37.

Phase 2 — Grid Search: Screened a 36-combination parameter space evaluating dimensions $d_{latent} \in \{64, 128, 256, 512\}$, weights $\beta \in \{0.5, 1.0, 2.0\}$, and learning rates $\eta \in \{10^{-3}, 5 \times 10^{-4}, 10^{-4}\}$. Models were limited to 15 epochs (patience 3) to isolate the Top-6 configurations via the lowest validation reconstruction loss.

Phase 3 — Selection via FID: The Top-6 models were retrained for 20 epochs (patience 4). Using the Fréchet Inception Distance (FID) over 1040 samples, the winning setup was established at $d_{latent} = 128$, $\beta = 0.5$, and $\eta = 5 \times 10^{-4}$, minimizing the distance to 262.43.

Phase 4 — Optimal Traditional Training: Long-form training of the optimal parameters proceeded for 60 epochs (patience 7). Early stopping triggered at epoch 46, restoring the peak checkpoint from epoch 40 (loss: 744.63).

Phase 5 — Improved Perceptual VAE: Maintained core optimal parameters but introduced a pre-trained VGG-16 feature extractor ($\lambda_{vgg} = 0.5$) and a linear KL Annealing scheduler scaling β from 0.0 to 0.5 over the first 15 epochs. Training finished at epoch 60, peaking at epoch 59.

Table 1. Complete optimization and hyperparameter mapping for all training phases.

Hyperparameter	Phase 1	Phase 2 (Grid)	Phase 3 (FID)	Phase 4 (Final)	Phase 5 (Imp.)	Baseline CN
Batch Size	64	64	64	64	64	32
Optimizer	Adam	Adam	Adam	Adam	Adam	Adam
Learning Rate (η)	10^{-3}	Variable	Variable	5×10^{-4}	5×10^{-4}	1×10^{-3}
Latent Dim. (d_{latent})	128	{64..512}	{128..512}	128	128	N/A
KL Weight (β)	1.0	{0.5..2.0}	0.5	0.5	$0.0 \rightarrow 0.5$	N/A
VGG Weight (λ_{vgg})	N/A	N/A	N/A	N/A	0.5	N/A
Max. Epochs	40	15	20	60	60	80
Early Stop (Patience)	5	3	4	7	7	5
Selection Metric	Val Loss	Val Recon	Val FID	Val Loss	Val Loss	Weighted F1

4.3 Evaluation Metrics

Classification performance is quantified using top-1 Accuracy and a Weighted F1-Score, which addresses the biological class imbalance across the 75 Lepidoptera categories by balancing precision and recall based on class support [?]. Validation confusion matrices visually track morphological classification errors. Generative quality is assessed via three criteria: the Structural Similarity Index Measure (SSIM) to quantify local edge and luminance retention in reconstructions; the Fréchet Inception Distance (FID) to measure multivariate Gaussian alignment between real and synthetic Inception-V3 features; and the Inception Score (IS) to evaluate conditional specificity and generated sample diversity.

5 Analysis of the Results

5.1 Quantitative Classifier and Generative Evaluation

To assess the augmentation efficacy of each generative paradigm, the **BaselineCNN** was trained under five distinct data scenarios while keeping all optimization parameters strictly static. Performance metrics on the validation set are compiled in Table 2, alongside the standalone generative evaluation metrics (FID, IS, and SSIM).

Table 2. Classifier performance under different augmentation scenarios and corresponding intrinsic generative model evaluation metrics.

Training Scenario	Acc. (%)	W. F1 (%)	Δ Acc.	FID ($\downarrow$)	IS ($\uparrow$)	SSIM ($\uparrow$)
Baseline (Raw Data Only)	43.65	40.22	Reference	N/A	N/A	N/A
Baseline + VAE Augmented	68.27	68.01	+24.62%	264.41	1.96 ± 0.08	0.4825
Baseline + CGAN Augmented	66.92	65.80	+23.27%	223.98	3.18 ± 1.29	0.0581
Baseline + DDPM Augmented	63.27	62.79	+19.62%	204.43	4.56 ± 0.67	0.0561
Baseline + Improved VAE	61.92	61.49	+18.27%	401.02	1.50 ± 0.09	0.1636

6 Conclusion

References

1. Author, F.: Article title. *Journal* **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) *CONFERENCE 2016, LNCS*, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). <https://doi.org/10.1007/1234567890>
3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25